Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 6

Name: Soham Kulkarni
PRN: 124B1F056
Subject: Design and Analysis of Algorithms
Batch: A3
GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

Select essential earthquake-relief packages for a truck of capacity W kg. Each item has a weight and utility value. Use 0/1 Knapsack to maximize utility without exceeding W: take each package once or leave it; do not split it.

2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

3. Course Outcome

- Apply dynamic programming to an exact single-truck resource-allocation problem.
- Reconstruct selected packages and compare DP with brute force and greedy selection.

Theory

0/1 Knapsack makes two choices for each package: include it or exclude it. The table dp[i][w] stores the highest utility using the first i items and capacity w. Utility is an importance score, not a price. Using the previous row when taking an item prevents selecting it more than once.

If an item is too heavy, leave it. Otherwise, compare the previous answer with the item's utility plus the best answer for the remaining capacity. The simple program handles one truck. Higher utility can encourage critical supplies, but it does not force their inclusion.

Working of the Algorithm

- 1. Initialize the DP table to zero for an empty selection.
- 2. For each item and capacity, compare taking the item with leaving it.
- 3. Trace the completed table backwards to recover the chosen items.
- 4. For changing supplies, revise the input and rerun. Multiple trucks and strict perishable priority require extra rules.

Complexity Analysis

The table contains (N+1)(W+1) states, and each state needs constant work. Time is O(N(W+1)), usually written O(NW) for positive W. This is pseudo-polynomial in numeric capacity.

Time Complexity

Best Case: O(N(W+1)).

Average Case: O(N(W+1)).

Worst Case: O(N(W+1)); all table states are filled.

Space Complexity

O(N(W+1)) for the complete table and selected-item reconstruction. Value-only DP can reduce table space to O(W), but this version keeps reconstruction simple.

Algorithm - Steps

- Step 1: Read item count, truck capacity, and each item's name, weight and utility.
- Step 2: Fill the table using the take-or-leave decision.
- Step 3: Backtrack and print selected items, used weight and maximum utility.

Pseudocode

```
dp[0][w] = 0 for every capacity w
for item i from 1 to N:
    for capacity w from 0 to W:
        dp[i][w] = dp[i-1][w]
        if weight[i] <= w:
            take = utility[i] + dp[i-1][w-weight[i]]
            dp[i][w] = max(dp[i][w], take)
backtrack from dp[N][W] to identify selected items
```

Flow of Execution

START -> Read packages and W -> Fill DP table -> Recover selected items -> Print result -> END.

6. Example

Truck capacity = 5 kg. Medicine: weight 2, utility 30; Food: weight 3, utility 40; Blankets: weight 4, utility 50. Medicine + Food weighs 5 kg and gives maximum utility 70. Medicine + Blankets weighs 6 kg and cannot be loaded.

Brute force takes O(2^N) time; ratio greedy takes O(N log N) but is not always exact for 0/1 items. With W=50 and items (10,60), (20,100), (30,120), greedy gives 160 while DP gives 220. For several trucks, sequential planning is a heuristic; exact joint planning needs additional capacity dimensions.

Conclusion

0/1 Knapsack DP gives an exact maximum-utility single-truck selection while respecting the weight limit. The take-or-leave table makes the result easy to explain. Priority and multiple-truck policies must be stated explicitly when extending the model.